

GetFit-Milestone 1

Members: Aluse Zhou, Harman Bhogal, Gurjot Mahay, & Jagraj Gill

API Features:

API → Open Weather API:

1. Allow the user to plan their workout based on the weather
 - a. Goal: Allow the user to check the weather and plan their workout accordingly.
 - b. Benefit: When the weather is bad or not ample some people do not really want to go out. However, if they have a workout based on the weather, they are more likely to work-out because they have the ability to adapt.
2. Clothing and Gear Suggestion
 - a. Goal: Based on the weather the user will be gives a list of possible clothing recommendations.
 - b. Benefit: We want to user to be most comfortable when working out. If the weather is rainy, we want to user to wear appropriate clothing and if it is sunny we want them to wear something appropriate for that climate.
3. Motivational messages:
 - a. Goal: Get the current weather of the user's location and deliver a tailored message based on the weather.
 - b. Benefit: Users receive encouragement that not only boosts their motivation but also acts as a warning to the user about the current weather in their local.

API → Wger API

1. Exercise Look Up
 - a. Goal: Allow the user to look up exercises that fit into their workout
 - b. Benefit: We want to user to be able to look at exercises that fit into their workout. Why? We want the user to be able to build there workout, achieve their goal, and have a good experience with their fitness journey.

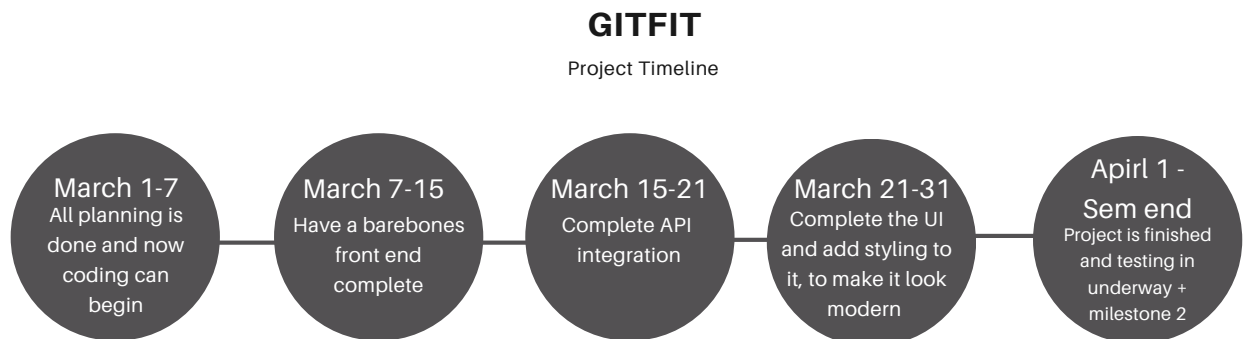
2. Workout builder

- a. Goal: The user can select exercises and add them to their workout
- b. Benefit: The user should be able to build a workout according to their needs. Time, New, Scared? These are all problems we plan to solve. We will offered tailored workout or the user can make their own.

3. Exercise detailed view

- a. Goal: For advanced users we want them to have a breakdown of what muscle groups are biased.
- b. Benefit: Advanced users can have a better view of what muscles group are worked. This allows them to better optimize their workouts and have a more structure to it.

Project Timeline:



March 1-7:

- At the time of writing this, this is the current week. We will complete milestone 1 and finally put the project into motion

March 7-15:

- For this week we have to have completed a bare bones front end, no need for css or any styling at this point in time.
- This includes all pages of the project

March 15-21:

- Now that the project front-end is done we now want to integrate the API's
- By the end of this week, we want to have at least one API fully functional and the second one almost complete

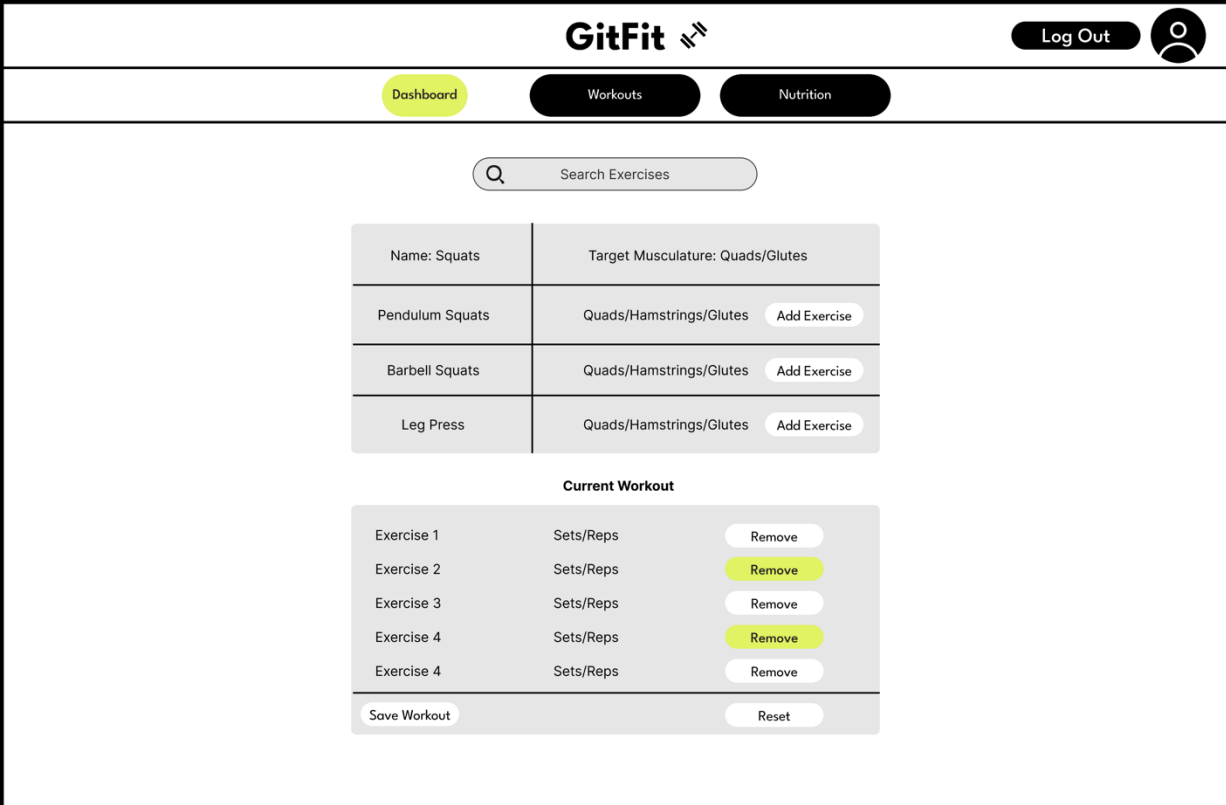
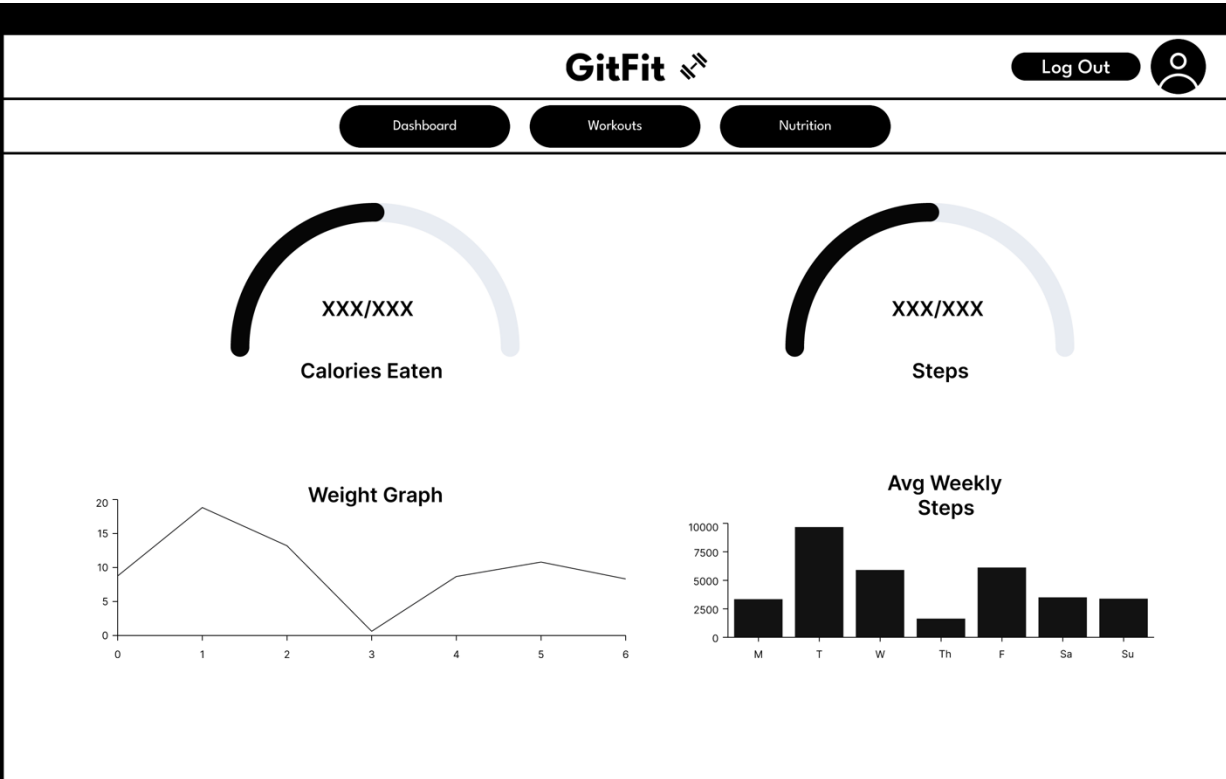
March 21-31:

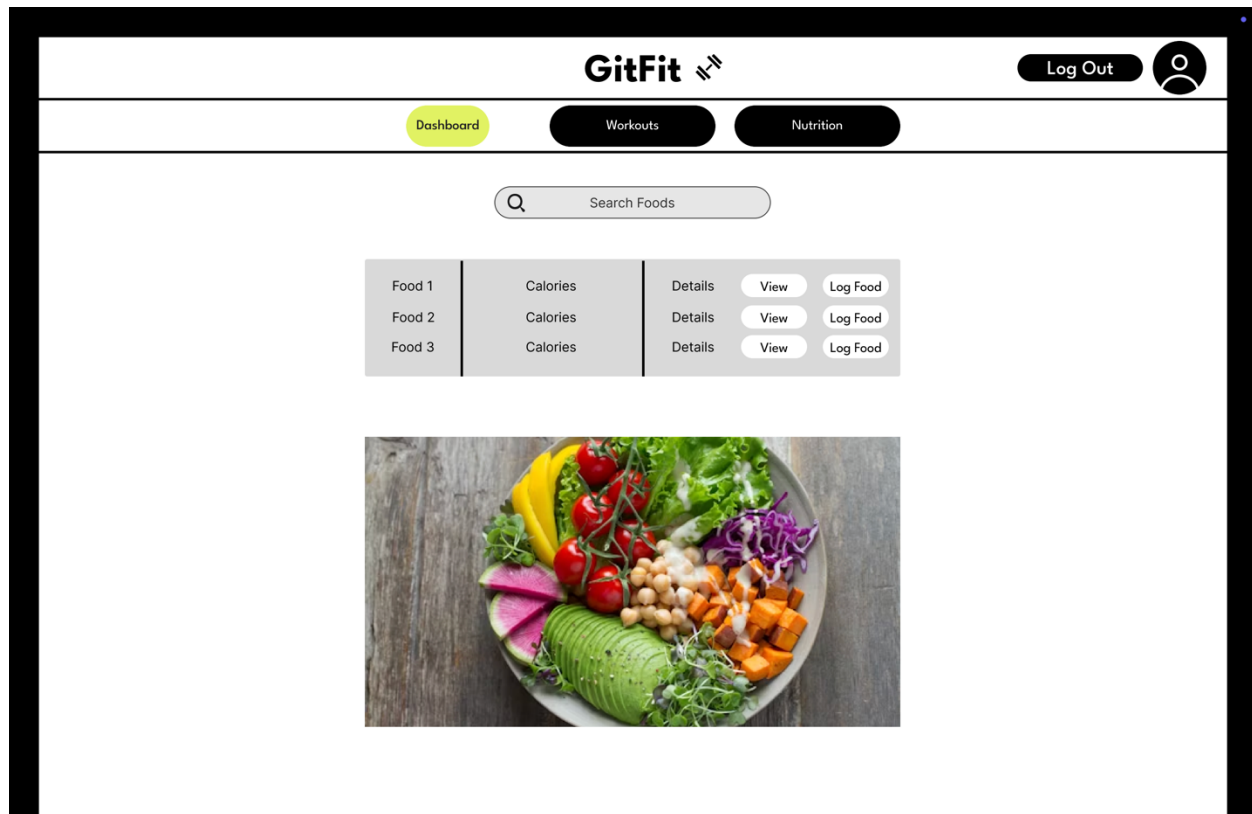
- Now that the website's functionality CSS styling will be added
- Make the website have a good UI/UX

March 31 – Semester END

- The coding project is complete and now testing is underway
- Rigours testing and trying to break and make the project act up in any way possible
- Test cases will be written prior to the start of this week

Mid-Fidelity Prototype(LINK):





SDLC METHOD:

The method we chose is a form of agile. Instead of daily stand-ups we chose to have daily check in report that answered the question, What did you do, Did you have any problems, What are you goals for today, and Will you need any help or extra time? Additionally, we decided on having a weekly meeting(on Sundays) to discuss what we did for the week, any issues, and other project related issues.

Risk Assessment:

Low-Risk Issues & Mitigation Strategies

1. Inconsistent UI Due to Multiple Developers Working on Different Elements:

Risk: With multiple developers working on different sections of the UI, there is a significant chance of inconsistencies in design. This can result in improper spacing, and layout issues that negatively impact the user experience.

Mitigation: To minimize this risk, it is essential to define clear design guidelines. These guidelines should include standard font sizes, button styles, and spacing rules, ensuring a cohesive look throughout the app. Additionally, conducting regular reviews of each team member's work before merging code can help maintain consistency.

2. Grammatical Errors on the Website:

Risk: With several people working on the project and a significant amount of text to manage, there is a possibility of grammatical errors slipping through.

Mitigation: Regular proofreading sessions should be incorporated into the development process. Additionally, writing in spell-checking software, such as MS Word, before adding the text to the code file will help catch errors early.

3. Lack of Clear Reasoning Behind Exercise Recommendations:

Risk: When exercise recommendations are influenced by factors like weather, users may not understand why a specific exercise has been excluded from their workout plan. This lack of clarity could confuse users or lead to dissatisfaction with their workout recommendations.

Mitigation: Every time an exercise is modified based on weather conditions or other criteria; the user should receive a notification explaining why the change was made.

4. **Icons and Apps Without Titles May Be Confusing:**

Risk: Icons such as the user profile, settings, or other interactive elements might not be immediately clear to users if they don't have accompanying titles. This could create confusion about what each icon represents.

Mitigation: Tooltips should be implemented to appear when a user hovers over an icon. This will clearly inform users what each icon does without cluttering the interface.

5. **Data Privacy Concerns from Users:**

Risk: Users may be concerned about how their personal workout data is used.

Mitigation: Before the user must enter sensitive info, they will be prompted with a message indicating that their data will not be used in any harmful way.

Medium-Risk Issues & Mitigation Strategies

1. Weather-Based Workout Recommendations are not always Relevant:

Risk: The app's weather-based workout suggestions might not always be relevant to all users. For instance, a user might prefer an indoor workout even though the weather on the outside isn't suitable.

Mitigation: To reduce this risk, the app should allow users to manually adjust the workout recommendations. By providing an option to override weather-based suggestions.

2. Inaccurate Info from User:

Risk: Users might input inaccurate information, such as unrealistic weights or incorrect activity levels, leading to flawed workout suggestions and inaccurate tracking of their progress.

Mitigation: The app should implement validation checks to ensure entered data falls within reasonable ranges. If it does not, the user should have to validate the info to ensure it was not a typo.

3. Overwhelming User Interface for New Users:

Risk: New users might find the websites interface overwhelming, especially if they are unfamiliar with how to navigate it or where to start. This could lead to frustration and a negative experience.

Mitigation: To address this, in the user's profile page, there should be a section providing suitable info on how the user should get started with using the website.

4. Outdated API Info:

Risk: The workout Api may display incorrect outdated exercises or have other wrong info.

Mitigation: The website should allow the user to flag certain exercises. These flagged exercises will then be excluded from being added to workouts.

5. User Frustration with Slow Website Load:

Risk: Users may experience delays when changing their workout plans, which could result in a slower website load. This may lead to decreased satisfaction and engagement.

Mitigation: To reduce this risk, a loading spinner or visual indicator should be displayed whenever the site is processing changes or loading new data. This will inform users that the website is working and reduce frustration.

High-Risk Issues & Mitigation Strategies

1. Major API Failure (Workout/Weather API Goes Down for Extended Time):

Risk: If the primary workout or weather API becomes unavailable for an extended period this could break certain key features. This could leave users unable to get

workout suggestions or weather-based recommendations, severely impacting the website's usability.

Mitigation: The website should implement a warning system that notifies users when certain functions are unavailable due to API failure.

2. Overlooking Health Conditions:

Risk: Users with health conditions such as heart conditions may follow a workout plan and cause damage to themselves due to their respective condition.

Mitigation: Before a workout plan is created, a pop-up should alert users that if they have a medical condition, it is best to consult a doctor before following the workout plan.

3. Outdoor Activity Hazards:

Risk: Users doing outdoor workouts such as running outside may face poor environmental hazards such as poor road conditions.

Mitigation: Provide users general safety tips for any outdoor activities so that users are adequately prepared to face any hazardous situations such as poor road conditions.

4. Inadequate User Support:

Risk: In case of a situation, where a user encounters an issue and the website has not accounted for a very specific niche issue, the user may abandon the website.

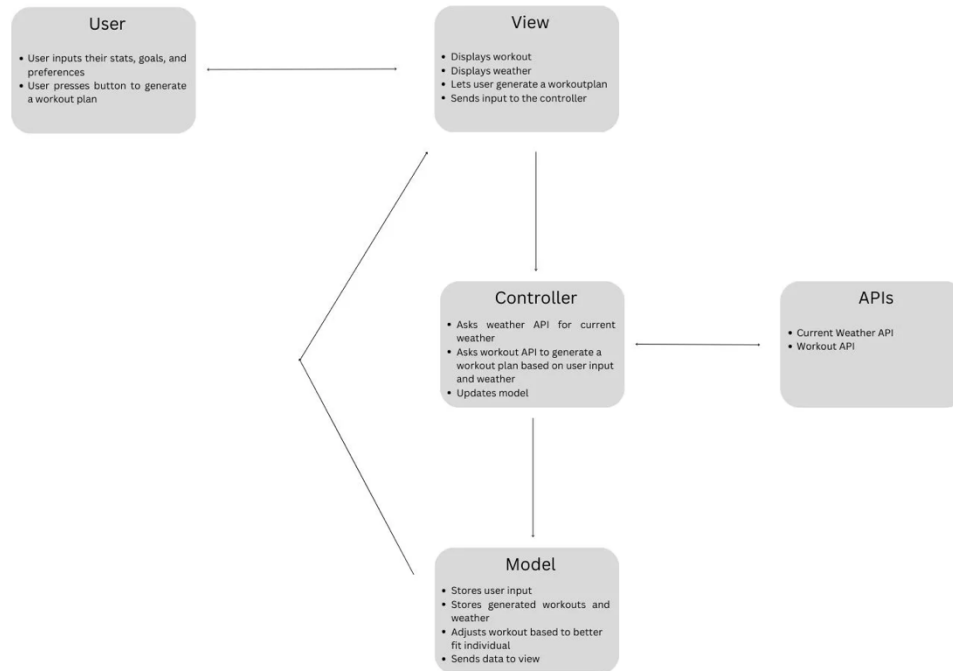
Mitigation: To mitigate this issue, a help section will be implemented, with a way to contact support through email, allowing users to submit feedback.

5. **Lack of Info on other areas of Fitness:**

Risk: A user may follow workout plans and may not see any progress visually due to the proper use of nutrition, and rest days. Users may blame the website for proper lack of progress and abandon the website.

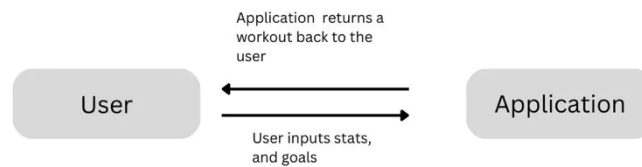
Mitigation: To mitigate this issue, links to additional text on nutrition and rest days can be found on the website to allow users to understand fitness in general better. Using this newfound info and the workout planner, users can see steady visual progress.

MVC Diagram:

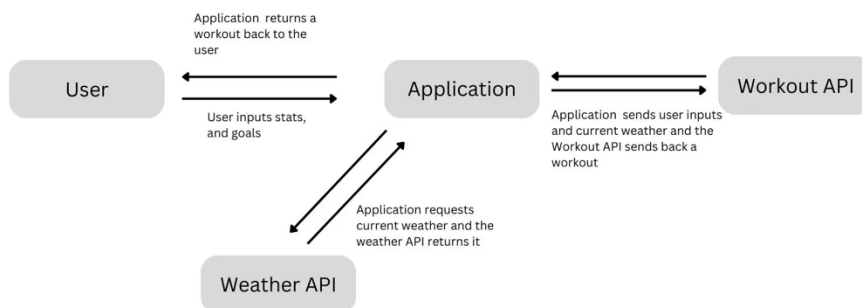


DATA FLOW DIAGRAM:

Level 0:



Level 1:



Work Breakdown Structure:

Task #	Task
1	Begin coding Barebones Frontend Development
1.1	Setup React.js Project
1.1.1	Initialize React.js project
1.2	Set up basic file structure
1.2.1	Create a component subfolder for reusable UI elements such as icons
1.2.2	Create a pages subfolder for main pages
1.2.3	Create an assets subfolder for images and static files
1.2.4	Set up basic CSS style files
1.3	Agree on CSS styling guidelines by putting guidelines into text file
1.4	Develop core pages with minimal UI
1.4.1	Create the Dashboard Page
1.4.1.1	Add in empty weight and avg weekly step graphs
1.4.1.2	Add in empty placeholder boxes for calories burned and steps taken
1.4.2	Create the Workouts Page
1.4.2.1	Add in non-functional search bar
1.4.2.2	Add in UI tables for current workouts, specific exercise info and add exercise buttons
1.4.3	Create the User page
1.4.3.1	Add in user profile picture on top of page with all elements below page.
1.4.3.2	Add in fillable user info text boxes.
1.4.3.3	Add in basic getting started section
1.4.3.4	Add in additional fitness info section
1.4.3.5	Add in support section
1.5	Implement basic navigation
1.5.1	Create navbar and footer components.
1.5.2	Create static user profile logo in top right
1.5.3	Make navbar and profile logo interactive by linking pages

Task #	Task
2	Implement API
2.1	Integrate Wger
2.1.1	Set up Search feature
2.1.1.1	Setup API connection
2.1.1.2	Implement exercise lookup functionality for search feature
2.1.1.3	Implement specific feature info about exercise(muscle group)
2.1.1.4	Handle errors and edge cases if no exercises by user are found
2.1.2	Enable workout building functionality
2.1.2.1	Set up basic UI elements such as exercise name, sets and reps
2.1.2.2	Implement changeable Workout builder, allowing new exercises to be added or deleted
2.1.2.3	Implement ability to take exercise from search area into workout builder
2.1.2.4	Allow save system, to save workout locally
2.1.2.5	Implement system recommendations
2.1.2.5.1	Take info from user profile page and find suitable exercises heeding to the specific constraints by user
2.1.2.5.2	Allow user to put exercises from suggestions into workout builder
2.1.2.5.3	Implement checking workout builder to indicate to user through notifications if any exercise violates any user constraints
2.2	Implement Weather Api
2.2.1	Gather weather info
2.2.1.1	Set up API connection
2.2.1.2	Fetch user location
2.2.1.3	Based on user location fetch current weather data
2.2.1.4	Have a UI icon and text on navbar to describe weather
2.2.2	Provide weather-dependent exercise recommendations
2.2.2.1	Define weather based rules for filtering exercises
2.2.2.2	Filter exercises based on weather rules and implement into suggestions
2.2.2.3	Allow user an override toggleable button on workout page to ignore weather settings

Task #	Task
3	Implement Risk Mitigation
3.1	Implement Constraint checking for user data
3.2	Implement notifications and pop-ups, warning user about system activity
3.2.1	Implement generic notification system for future use
3.2.2	Data privacy notification
3.2.3	API Failure notification
3.2.4	System exercise modifications to workout builder notification
3.2.5	Health-Warning notification
3.3	Loading spinning indicator for workout builder load
3.4	Flag system for outdated or disliked exercises
3.4.1	Interactable flag buttons on the exercises from the search function
3.4.2	Options to unflag exercises in user profile page
3.4.3	Flagged exercises do not appear in search
3.5	Safety Info found on Workout Page
3.6	Determine edge cases and possible tests cases for future testing

Task #	Task
4	Styling and Completing UI
4.1	Apply tailwind CSS styling
4.1.1	Style Dashboard
4.1.2	Style Workouts Page
4.1.3	Style User Profile
4.2	Enhancements to UI
4.2.1	Tooltips for UI elements
4.2.2	Style navbar and footer elements
4.2.3	Add animations and effects
4.2.3.1	Add hover effects for key UI elements
4.2.3.2	Add animations for graphs, and adding exercises
4.2.3.3	Add animations to notification system
4.3	Finalize visual design
4.3.1	Add branding elements such as logos
4.3.2	Define color scheme for background, and text elements
4.3.3	Enable graphs uto use data from profile page to display info

Task #	Task
5	Project Testing and Deployment
5.1	Final testing
5.1.1	Conduct testing for React components
5.1.2	Test Api integrations and error handling
5.2	Bug fixing for any problems found previously
5.3	Deploy product

Appendix:

Group Contribution:

1. API Feature list: Jagraj Gill
2. Mid-Fidelity prototype: Gurjot Mahay
3. WBS Structure + Added to github: Harman Bhogal
4. Project timeline: Jagraj Gill
5. Risk Assessments: Harman Bhogal
6. Data flow diagram: Aluse Zhou
7. MVC diagram: Aluse Zhou

Changelog:

The project has slightly changed. Due to the nature of the Food Nutrition API, it would not be compatible with our project as it needs a backend to function. Instead of using this Food Nutrition API we have decided to use the OpenWeather API. Now based on the weather the user will be given a workout, clothing suggestions, and they are still able to program their workout.